

A Comparative Benchmark: Python versus Rust on Numerical Solving of the 2-D Heat Equation

1. Problem and Numerical Method

The transient 2-D heat equation on a unit-square plate is:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad (x, y) \in [0, 1]^2, \quad t \in [0, t_{\text{end}}]$$

with $\alpha = 1.172 \times 10^{-5} \text{ m}^2/\text{s}$ (thermal diffusivity of structural steel). Boundary conditions are Dirichlet ($T = 0$ on all four edges, held fixed). The initial condition is a centred hot square occupying 20 % of the plate ($T = 100$).

The equation is solved with the explicit forward-in-time, centred-in-space (FTCS) finite-difference scheme:

$$T_{i,j}^{n+1} = T_{i,j}^n + r_x (T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n) + r_y (T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n)$$

where $r_x = \alpha \Delta t / \Delta x^2$ and $r_y = \alpha \Delta t / \Delta y^2$. The Von Neumann stability analysis requires $\alpha \Delta t (\Delta x^{-2} + \Delta y^{-2}) \leq 1/2$, giving the maximum stable time step:

$$\Delta t_{\text{max}} = \frac{\Delta x^2 \cdot \Delta y^2}{2\alpha(\Delta x^2 + \Delta y^2)}$$

A CFL factor $C = 0.5$ is applied so that $\Delta t = C \cdot \Delta t_{\text{max}}$. The number of steps is $n_t = \lceil t_{\text{end}} / \Delta t \rceil$, rounded up so that the simulation covers exactly t_{end} .

2. System Details

Parameter	Value
Hardware model	Lenovo IdeaPad 330-15IKB
Processor	Intel Core i3-6006U @ 2.00 GHz
Physical / logical cores	2 / 4
NUMA nodes	1
L1d / L1i (per core)	32 KiB / 32 KiB
L2 (per core)	256 KiB
L3 (shared)	3 MiB
RAM	8 GiB DDR4-2133
Operating system	Ubuntu 24.04.4 LTS (64-bit)
Kernel	Linux 6.17.0-35-generic
Python	CPython 3.12.12
Rust	1.93.0 (stable)

Table 1: Hardware and software environment.

Package	Version
numpy	2.4.2
h5py	3.16.0
typer	0.24.1

Crate	Version
ndarray	0.15.6
hdf5	0.8.1
clap	4.6.1
anyhow	1.0.103
serde	1.0.228
toml	0.7.8

Table 2: Major third-party (direct) dependencies managed via uv and cargo. Versions for other dependencies may be viewed in project uv and cargo lockfiles.

3. Parallel Fraction and Theoretical Limits

3.1. Sequential and Parallel Fractions

At each time step the only inherently sequential operations are re-pinning the four Dirichlet boundary edges and inter-worker synchronisation. All interior stencil updates are data-parallel. For a grid of $n_x \times n_y$ points:

$$p_{\text{ideal}} = \frac{(n_x - 2)(n_y - 2)}{n_x n_y} = \frac{510 \times 510}{512 \times 512} \approx 0.9922 \Rightarrow s_{\text{ideal}} = 1 - p \approx 0.78\%$$

The sequential fraction consists of $2n_x + 2n_y - 4 = 2044$ boundary assignments per step — less than 0.8 % of the 262,144 total cell updates.

3.2. Amdahl's Law — Strong Scaling

For a fixed problem size, Amdahl's law [1] gives the theoretical speedup achievable with n workers:

$$S_{\text{Amdahl}}(n) = \frac{1}{s + p/n}$$

As $n \rightarrow \infty$, speedup is bounded by $S_{\text{max}} = 1/s$. Using the ideal parallel fraction:

$$S_{\text{max}}^{\text{ideal}} = \frac{1}{1 - 0.9922} = \frac{1}{0.0078} \approx 128$$

In practice, synchronisation overhead and memory-bandwidth contention reduce the effective p below 0.9922 (see Section 5.1 and Section 5.2).

3.3. Gustafson's Law — Weak Scaling

Gustafson's law [2] addresses weak scaling, where the per-worker problem size is held constant while the number of workers grows. The scaled speedup is:

$$S_{\text{Gustafson}}(n) = p \cdot n + (1 - p)$$

Ideal weak scaling gives $S(n) = n$ (100 % efficiency). The sequential fraction $s = 1 - p$ limits how closely this ideal can be reached.

4. Implementations

All four variants produce numerically identical results: the maximum pairwise difference in the final temperature field across all implementations and worker counts is 1.99×10^{-13} (floating-point rounding noise).

- **heatpy (serial Python)** — NumPy vectorised stencil via `numpy.roll`; Dirichlet boundary re-pinned after every step.
- **heatpy-mp (parallel Python)** — Interior rows partitioned into contiguous bands, one per worker process. Workers share two complete frame buffers via `multiprocessing.shared_memory.SharedMemory`, so halo rows are read directly from shared memory without message passing. A single `multiprocessing.Barrier` per step synchronises all workers.
- **heatrs (serial Rust)** — Explicit stencil loop with a pointer-swap double buffer (`std::mem::swap`); compiled with full optimisations (`--release`), enabling auto-vectorisation to SIMD instructions.
- **heatrs-mt (parallel Rust)** — Row-band decomposition using `std::thread::scope` and two `Arc<UnsafeCell<Vec<f64>>>` double buffers. Two `std::sync::Barrier` instances per step — one after

the stencil compute and one after snapshotting — guarantee race-freedom without a mutex on the shared buffer.

5. Scaling Experiments

Each configuration was repeated 5 times; reported times cover only the solver loop (file I/O excluded). Outliers are defined as runs deviating by more than three standard deviations from the mean. Measurements were collected for 1, 2, and 4 workers/threads (matching the two physical / four logical cores of the test machine); a 3-worker configuration was not run, so it is omitted from all tables and figures below.

For weak scaling the grid grows as $n_x(n) = 256\sqrt{n}$, $n_y(n) = 256\sqrt{n}$, so that work per worker is proportional to $256^2 = 65,536$ cells per time step — constant regardless of worker count. However, the CFL-limited time step $\Delta t \propto \Delta x^2$ shrinks as the grid is refined, so the number of steps n_t needed to reach the same physical end time t_{end} grows linearly with n (see Table 3). Weak scaling therefore does not hold total work constant here: each worker still does a constant amount of work per step, but the number of steps increases with the worker count, so wall-clock time is expected to grow with n even under perfect parallel efficiency.

Workers	Grid	n_t
1	256×256	308
2	362×362	615
4	512×512	1229

Table 3: Weak-scaling grid sizes ($t_{\text{end}} = 50$ s). Total cells per step $\approx 65,536 \times n_{\text{workers}}$, but n_t grows roughly linearly with n_{workers} because of the CFL constraint.

5.1. Strong Scaling — Python

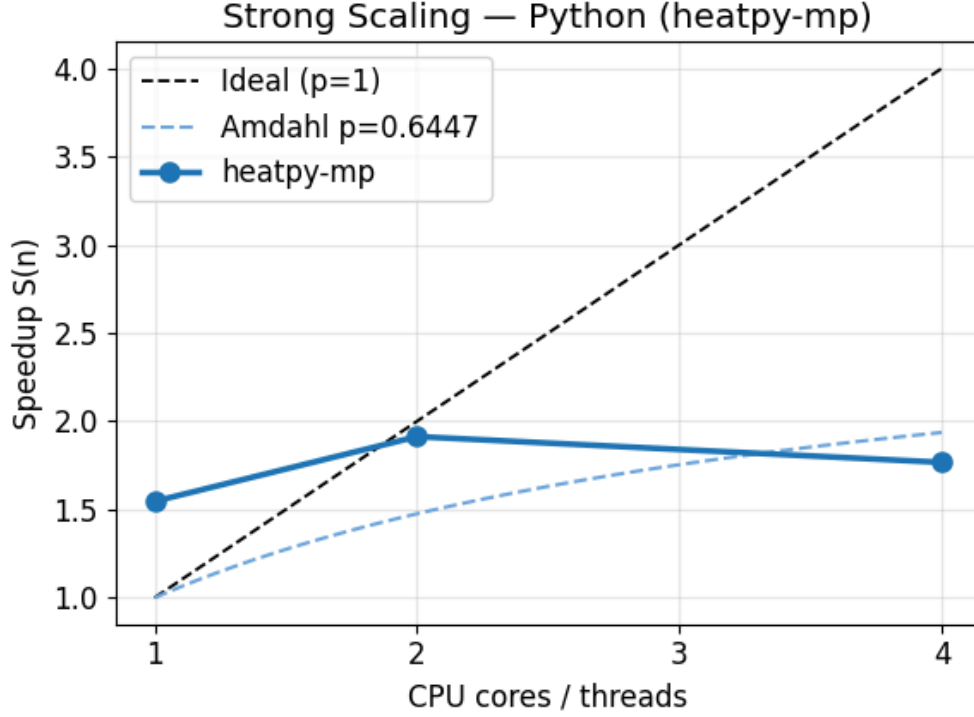


Figure 1: Strong scaling of *heatpy-mp* relative to the serial *heatpy* baseline.

Grid: 512×512 , $t_{\text{end}} = 50$ s, $n_t = 1229$.

Dashed blue line: Amdahl’s law with empirically fitted $p = 0.6447$.

Workers	Mean (s)	Std (s)	Speedup	Outliers
heatpy (serial)	9.9403	0.1195	1.000	0
1 (heatpy-mp)	6.4296	0.0931	1.546	0
2	5.1964	0.0434	1.913	0
4	5.6256	0.1461	1.767	0

Table 4: Strong-scaling results for Python. $\text{Speedup} = T_{\text{serial}}/T_n$.

The parallel variant already gains $1.55\times$ with a single worker and peaks at $1.91\times$ with two workers, before falling back slightly to $1.77\times$ at four. The gain at one worker arises from a structural difference between the two implementations: *heatpy* uses `numpy.roll`, which allocates a temporary array on every call, whereas *heatpy-mp* operates on contiguous shared-memory row slices without intermediate allocation. The regression from two to four workers is consistent with the machine’s topology: the test system has only two physical cores exposing four logical cores via hyper-threading (Table 3 and the environment table both reflect this). Workers three and four land on sibling hyper-threads of the same two physical cores, which

share L1/L2 caches, the 3 MiB L3, and the DDR4 memory bus with their sibling, so they add scheduling and synchronisation overhead without adding independent compute or memory bandwidth.

Fitting Amdahl’s law to the measured values gives an effective parallel fraction $p_{\text{eff}} = 0.6447$ and a theoretical maximum of $S_{\text{max}} = 1/(1 - 0.6447) \approx 2.81 \times$. This is well below the ideal $p_{\text{ideal}} = 0.9922$: the fit is pulled down by the four-worker regression, reflecting that only two workers can run on genuinely independent cores on this hardware.

5.2. Strong Scaling — Rust

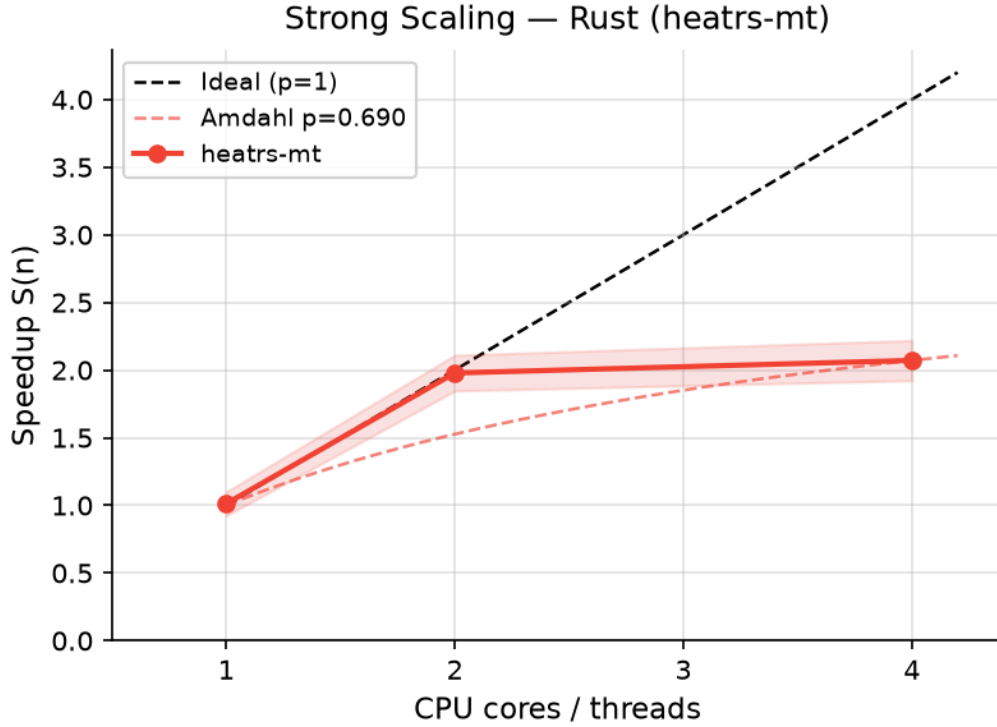


Figure 2: Strong scaling of *heatrs-mt* relative to the serial *heatrs* baseline.

Same grid as above.

Dashed red line: Amdahl’s law with empirically fitted $p = 0.7296$.

Threads	Mean (s)	Std (s)	Speedup	Outliers
heatrs (serial)	2.4909	0.0829	1.000	0
1 (heatrs-mt)	2.4610	0.1980	1.012	0
2	1.2589	0.0725	1.979	0
4	1.2025	0.0760	2.071	0

Table 5: Strong-scaling results for Rust. $\text{Speedup} = T_{\text{serial}}/T_n$.

The multi-threaded Rust solver shows essentially no gain at one thread ($1.01\times$, as expected — it is running the same partitioned algorithm on a single thread with added synchronisation), then scales to $1.98\times$ at two threads and continues improving to $2.07\times$ at four — unlike the Python variant, Rust does not regress on the two extra hyper-threads. The serial Rust solver at 2.4909 s is already $9.9403/2.4909 \approx 3.99\times$ faster than the serial NumPy implementation, consistent with auto-vectorised SIMD stencil code and zero-copy double-buffer management. That the thread-based Rust implementation keeps gaining through four logical cores, while the process-based Python implementation does not, suggests its per-step synchronisation (`std::sync::Barrier`, no allocation, no IPC) is cheaper relative to compute than Python’s `multiprocessing.Barrier` and shared-memory handshake, so it tolerates hyper-thread sharing of the L3 and memory bus better. The fitted effective parallel fraction $p_{\text{eff}} = 0.7296$ gives $S_{\text{max}} = 1/(1 - 0.7296) \approx 3.70\times$, noticeably higher than the Python fit, reflecting the continued gains through four threads.

5.3. Weak Scaling — Python

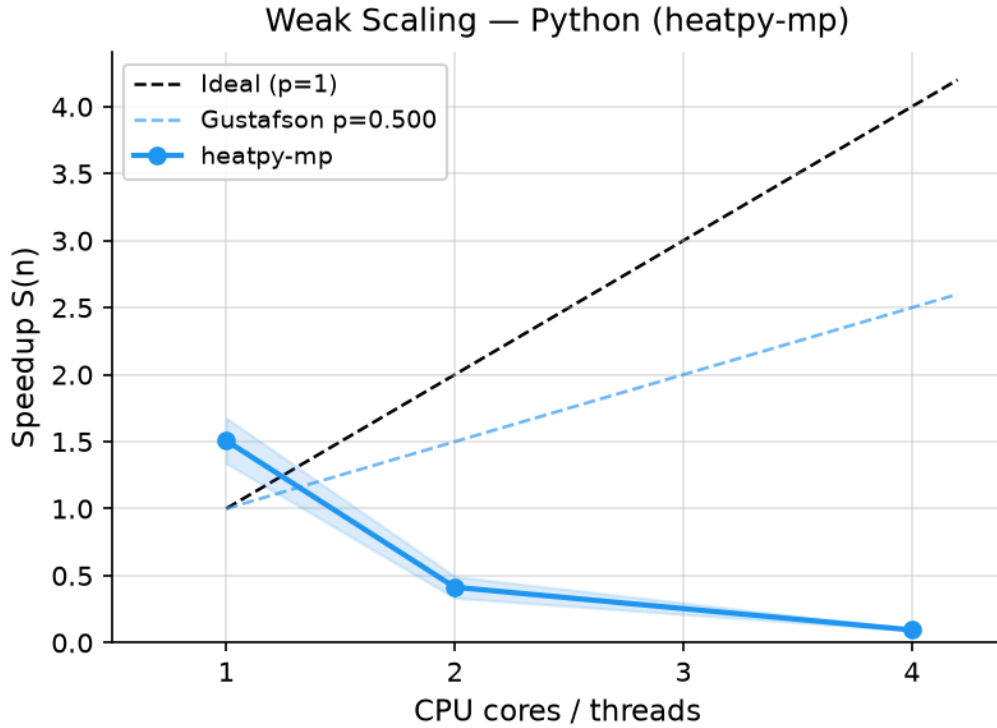


Figure 3: Weak scaling of *heatpy-mp*.

Scaled speedup $S(n) = n \cdot \eta$ where $\eta = T_1/T_n$ is the parallel efficiency.

Dashed blue line: Gustafson’s law fit, $p = -0.270$ (see discussion below).

Workers	Grid	Mean (s)	Std (s)	Efficiency η	Outliers
1	256×256	0.3610	0.0393	1.000	0
2	362×362	1.3155	0.2509	0.274	0
4	512×512	5.7568	0.2498	0.063	0

Table 6: Weak-scaling results for Python. Efficiency $\eta = T_1/T_n$.

Parallel efficiency collapses from 100 % at one worker to 6.3 % at four — much steeper than a synchronisation-overhead story alone would predict. As noted above, this experiment does not actually hold total work constant: n_t grows roughly linearly with the worker count (308, 615, 1229 steps for 1, 2, 4 workers) because refining the grid to keep cells-per-worker fixed also shrinks the CFL-limited time step. So even a perfectly efficient implementation would need noticeably more wall-clock time at $n = 4$ than at $n = 1$ here, simply because it is running roughly $4 \times$ as many time steps, not because of overhead. On top of that expected growth, the multiprocessing.Barrier synchronisation cost per step and the transition of the frame buffer from L3-resident (0.5 MiB at 256^2) to DDR4-resident (2 MiB at 512^2) further inflate T_n . Because the underlying workload is not actually constant, the Gustafson model is not a good fit here: the least-squares fit returns a nonphysical $p \approx -0.27$, which is itself a symptom that scaled speedup is falling faster than the model’s assumptions allow, rather than a meaningful “small parallel fraction” estimate.

5.4. Weak Scaling — Rust

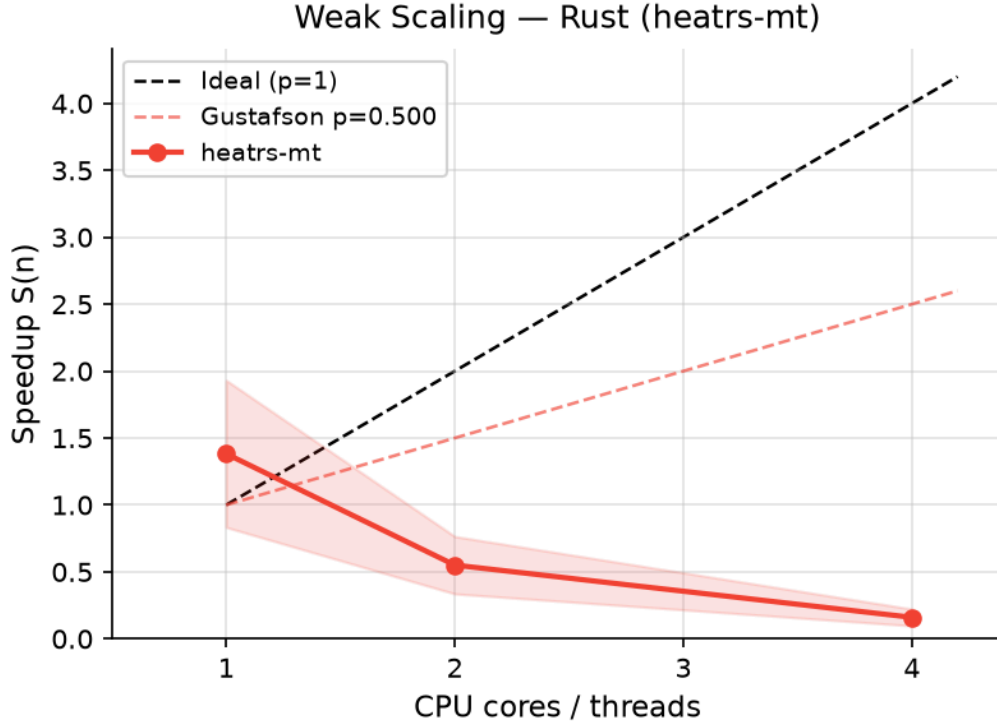


Figure 4: Weak scaling of heatrs-mt.

Scaled speedup $S(n) = n \cdot \eta$.

Dashed red line: Gustafson’s law fit, $p = -0.181$ (see discussion below).

Threads	Grid	Mean (s)	Std (s)	Efficiency η	Outliers
1	256×256	0.1480	0.0123	1.000	0
2	362×362	0.3715	0.0084	0.398	0
4	512×512	1.2775	0.0529	0.116	0

Table 7: Weak-scaling results for Rust. Efficiency $\eta = T_1/T_n$.

Rust weak-scaling efficiency falls to 39.8 % at two threads and 11.6 % at four — a smaller relative collapse than Python’s, though still well short of ideal. The same n_t -growth argument applies here: the workload is roughly $4 \times$ larger by step count at four threads than at one, so a substantial slowdown is expected on physical grounds alone, independent of synchronisation cost. The single-thread baseline is $0.3610/0.1480 \approx 2.44 \times$ faster than the Python counterpart at the same (smallest) grid, a more modest gap than the strong-scaling comparison because this grid is small enough that neither implementation is bandwidth-bound. As with Python, the Gustafson fit is not physically meaningful here ($p \approx -0.18$): it reflects that the measured workload was not actually held constant across the series, so scaled speedup falls faster than the model can represent with $p \geq 0$.

6. Summary

Metric	py serial	py-mp (4)	rs serial	rs-mt (4)
Mean time (s)	9.9403	5.6256	2.4909	1.2025
Speedup vs own serial	—	$1.77 \times$	—	$2.07 \times$
Rust vs Python speedup	$3.99 \times$	$4.68 \times$	—	—
Amdahl p_{eff}	—	0.6447	—	0.7296
Amdahl S_{max} (fitted)	—	$2.81 \times$	—	$3.70 \times$
Amdahl S_{max} (ideal)	—	$128 \times$	—	$128 \times$

Table 8: Summary at the strong-scaling grid (512×512 , $n_t = 1229$).

The ideal parallel fraction of the FTCS stencil is $p_{\text{ideal}} = 99.22 \%$, giving a theoretical Amdahl limit of $128 \times$ on an infinite-core machine. On this two-physical-core / four-logical-thread laptop the achieved speedup at four workers is $1.77 \times$ (Python) and $2.07 \times$ (Rust); Python actually peaks higher, at $1.91 \times$, with only two workers. Both implementations fall well short of the ideal limit because only two of the four logical “cores” are independent physical cores — the third and fourth workers are hyper-threads competing for the same execution units, L3 cache, and memory bus as their sibling. Effective parallel fractions fitted from measurements ($p_{\text{eff}} = 0.64$ for Python, 0.73 for Rust) are much closer to the ideal value than a purely bandwidth-bound story would suggest, but the Python fit is pulled down by its four-worker regression, while Rust’s thread-based implementation keeps gaining through four threads.

The serial Rust solver is $3.99 \times$ faster than serial NumPy on the same problem, owing to auto-vectorised SIMD stencil code and zero-copy double-buffer management. Both languages parallelise the FTCS scheme

correctly and produce results that agree to floating-point precision. The weak-scaling experiments, by contrast, did not hold total computational work constant (the CFL-limited step count grows with worker count as the grid is refined), so the large efficiency drop-off seen there partly reflects a growing workload rather than pure parallel overhead; a fairer weak-scaling protocol would fix n_t independent of grid size.

Bibliography

- [1] G. M. Amdahl, ‘Validity of the single processor approach to achieving large scale computing capabilities’, *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967, doi: 10.1145/1465482.1465560.
- [2] J. L. Gustafson, ‘Reevaluating Amdahl's law’, *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988, doi: 10.1145/42411.42415.
- [3] Wikipedia contributors, ‘Heat equation’. [Online]. Available: https://en.wikipedia.org/wiki/Heat_equation
- [4] Wikipedia contributors, ‘FTCS scheme’. [Online]. Available: https://en.wikipedia.org/wiki/FTCS_scheme
- [5] L. Xin, ‘Scalability: Strong and Weak Scaling’. [Online]. Available: <https://www.kth.se/blogs/pdc/2018/11/scalability-strong-and-weak-scaling>
- [6] NumPy developers, ‘NumPy 2.4.2’. [Online]. Available: <https://numpy.org/>
- [7] M. Liu, ‘Plotters — A Rust drawing library’. [Online]. Available: <https://github.com/plotters-rs/plotters>